



PROJECT HANDOVER · JULY 2026

Portfolio Allocation App

Status, architecture & how to run it on Teroxx cloud

Prepared by Aleksander Biesaga — Digital Assets, Teroxx

Handover to

Research — Jannick Broering, Leonardo Larieira

IT — Emil Jorgensen

How to read this deck

This one deck serves two audiences. Every slide is tagged so you can read only what is relevant to you — or read it all for the full picture.

FOR RESEARCH

What the app does, the day-to-day workflow, the model, and where it lives.
No code required.

FOR IT

Architecture, data, security, integrations, and a step-by-step runbook to redeploy on Teroxx-owned cloud.

WHAT'S INSIDE

- 1 What the app does** Capabilities, workflow, the model & data — plain language
- 2 Project status** What's live, known limitations, where it runs today
- 3 Architecture** Stack, data, security, the proposal engine, integrations
- 4 Standing it up on your cloud** Migration runbook, recommended path, credentials transfer
- 5 Ownership & handover** Who owns what, first-week checklist, contacts & index

1

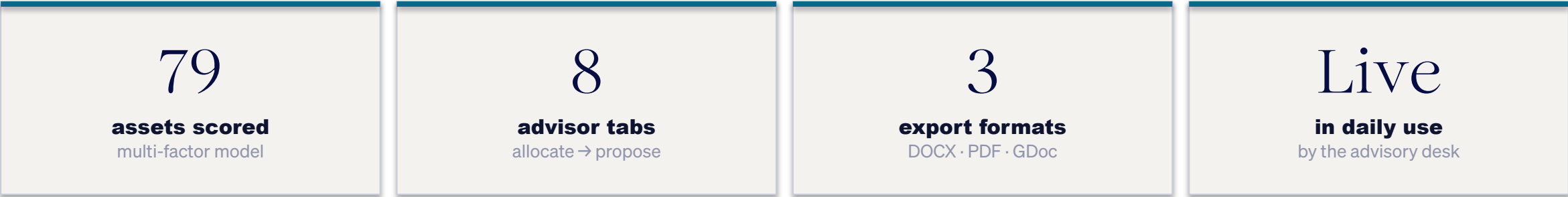
FOR RESEARCH

What the app does

- Capabilities & the advisor workflow
- The investable universes & baskets
- The allocation model
- Where the data comes from

In one minute

The Portfolio Allocation App is the web version of Teroxx's Portfolio Allocation Model (v5.0). An advisor picks a client's risk profile and investable universe; the app scores 79 crypto assets, builds a personalised allocation, tracks the live portfolio, and exports a fully branded client proposal as Word, PDF or Google Doc.



WHY THIS HANDOVER MATTERS

Today the app runs on Aleksander's **private personal Render account**. When that account goes away, so does the deployment. The goal of this handover is to move it onto Teroxx-owned cloud and Teroxx-owned credentials, with Research owning the content and IT owning the infrastructure.

The eight advisor tabs

1

Allocator

Risk profile + universe → target allocation table & donut chart

2

Your Portfolio

Enter a portfolio value → position sizes with BUY actions

3

Recurring Buys

DCA planner: scope, horizon, minimum order + backtest

4

Factor Scores

The 5-factor model: weights, radar & composite scores

5

Fundamentals

The 10-factor fundamental model per asset

6

Allocations

Risk tilts, strategic weights, per-asset breakdown

7

Rebalancing

Current vs target → concrete BUY / SELL actions

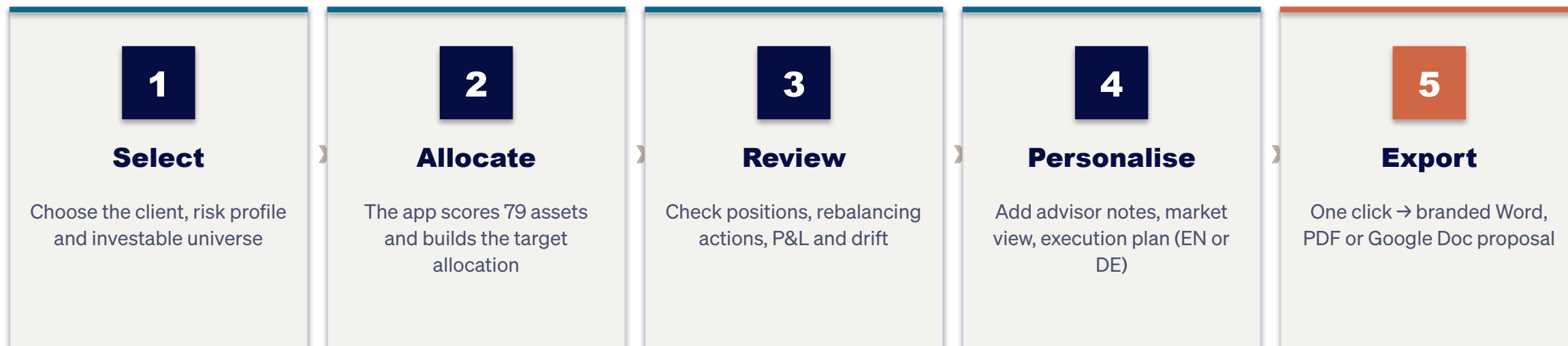
8

P&L Tracker

Entry price/qty → unrealised P&L against live prices

From risk profile to client proposal

The everyday advisor journey — five steps, ending in a branded document ready to send.



Faithful proposals: when the advisor hits export, the app freezes the exact on-screen allocation, so the document is a true record of what was reviewed — not a later recompute against moved market data. The Word file is the master; PDF and Google Doc are conversions of it, so the three can never drift apart.

The selectable universes & baskets

Before allocating, the advisor picks a starting list. Two kinds: the firm's **model portfolios** (broad, risk-tiered) or a single-theme **thematic basket** (index-style, 100% in-theme).

MODEL PORTFOLIOS — pick one

Terorxx Core <i>Live, active offering</i> USDC · EURC · PAXG · BTC · ETH · BNB · XRP · ADA · POL	9
Terorxx Expanded <i>Confirmed MVP next step</i> Core + MNT, and DeFi/infra: AAVE · UNI · COMP · EUL · PENDLE · SYRUP · ENA · ONDO · LINK · QNT · CHZ	21
Terorxx Extended <i>Full acquirable universe</i> Expanded + majors & research picks: SOL · HYPE · TRX · AVAX · DOT · ARB · SUI · RENDER · AKT · AR · VVV · AERO · LTC · BCH · DOGE ...	40

THEMATIC BASKETS — index-style, in-theme

11	DeFi Basket Lending, DEXs, perps & yield
9	Smart-Contract L1 Basket L1 platforms & settlement networks
4	AI & Compute Basket Decentralised AI, GPU compute, storage
2	RWA Basket Tokenised real-world assets & treasuries
3	Layer 2 Basket Ethereum scaling networks & rollups
2	Payments Basket Established P2P payment networks

Then choose: **risk profile** (Conservative · Balanced · Growth · Aggressive) × **allocation mode** (Standard = market-cap weighted · **Fundamental = factor-score weighted, the default**). The full research model scores 79 tokens; the 40-token union is what the firm can acquire today.

How the allocation is built

Every asset earns a score; scores drive tier weights; a macro overlay tilts risk up or down. All transparent and rules-based.

STEP 1 SCORE EACH ASSET

- Value-accrual signals: valuation (FDV/fees, FDV/TVL), fee revenue, dilution, buybacks
- Sector-tailored signals incl. price momentum & developer activity
- Every signal computed live from market & on-chain data

STEP 2 BUILD THE TIERS

- Assets sorted into strategic tiers (core → satellite)
- Risk profile sets how much sits in each tier
- Defensive sleeve (stablecoins + gold hedge) for ballast

STEP 3 OVERLAY THE REGIME

- 22-indicator macro composite scores the market 0–100
- Sentiment, TradFi, on-chain and technical signals
- Tilts overall risk toward the current regime

Result: a client-ready, risk-appropriate target allocation the advisor can explain line by line.

Where the numbers come from

The app pulls live market data from six external sources. Only two need an API key; the other four are public. Prices are cached to stay fast and within rate limits.

CoinMarketCap Spot prices, market cap, 24h change Auth: API key · 5 min cache	CoinGecko Fallback prices, dev activity, supply Auth: Demo key · up to 24h	Binance Futures Funding rates, open interest Auth: Public · 5 min cache
DeFiLlama Protocol TVL, fees, DEX volume Auth: Public · 2 h cache	Messari Network transactions & active addresses Auth: Public · 1 h cache	Macro composite Fear/greed, VIX, treasuries, BTC trend Auth: Public mix · hysteresis

Action for IT: CoinMarketCap and CoinGecko must run on **Teroxx-owned API keys** — the app currently relies on shared / personal defaults. The four public sources need no key.

If a source is briefly down, the app retries with backoff and falls back to cached values — it degrades gracefully rather than breaking.

2

RESEARCH + IT

Project status

- What is live and working
- Known limitations to watch
- Where it runs today

What's done, and what to watch

LIVE & WORKING

- ✓ **Allocation engine** — all 8 tabs, 79 assets, both scoring models
- ✓ **Client CRM** — clients, positions/lots, drift, scenarios, audit log
- ✓ **Proposal export** — DOCX / PDF / Google Doc, EN + DE, fully branded
- ✓ **Live market data** — 6 sources with caching and graceful fallback
- ✓ **Auth & audit** — session login; every change logged by user
- ✓ **Stable v1 API** — token-authenticated client endpoints
- ✓ **Deployed** — auto-deploys on every git push to main

KNOWN LIMITATIONS & GOTCHAS

- ! **Confirm the DB backend** — the app supports managed Postgres via DATABASE_URL; verify in the Render dashboard, and put the Teroxx deployment on managed Postgres (Section 4).
- ! **No automated test suite** — changes are verified manually in the UI.
- ! **Free-tier memory** — vector charts are off to avoid out-of-memory on 512 MB; PNG charts used instead.
- ! **Google Docs geo-blocks** — export must run from an allowed region; some servers get 403.
- ! **Secrets in repo config** — a few keys sit in render.yaml; rotate them on migration.

Where it lives today

One running deployment, two code repositories. The deployment is the single point of failure this handover resolves.

RUNTIME — action required

Render.com · free web-service tier

Hosted on Aleksander's **private personal account**. Docker image built from the repo; auto-deploys on push. This must move to a Teroxx-owned account/cloud.

SOURCE CODE — in good shape

GitLab (Teroxx) — IT-created group; Aleksander is maintainer

GitHub (private) — personal mirror, to be retired

Full codebase is on GitLab (**main** + import). IT owns deployment going forward.

TO KEEP THE APP RUNNING AFTER MIGRATION, FOUR THINGS MUST TRANSFER

1 Code

Already on Teroxx GitLab — nothing to move

2 A host

Any Docker-capable cloud (Section 4)

3 A database

A Teroxx-managed Postgres (RDS); verify current backend

4 Credentials

API keys, session secret, Google service account

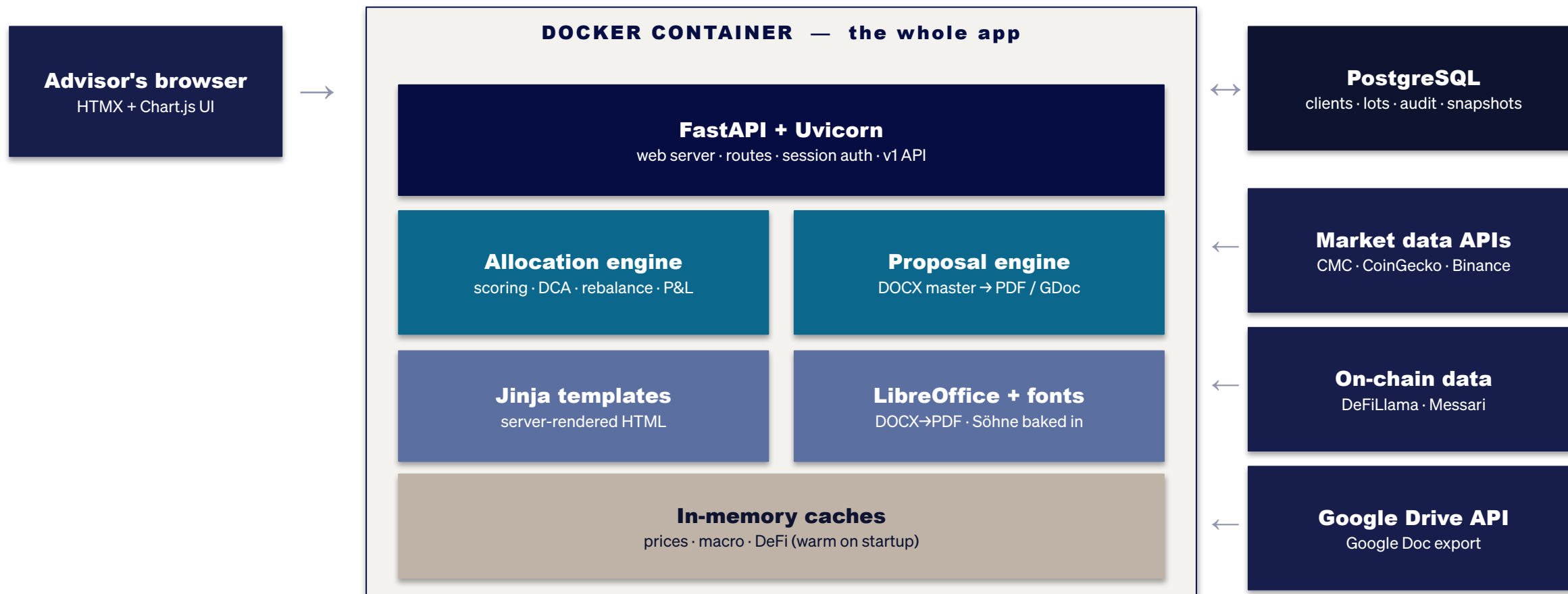
3

FOR IT

Architecture

- The stack & how it fits together
- Data, security & the proposal engine
- The environment-variable contract

How the system fits together



No dependencies on other internal Teroxx systems (confirmed with IT) — only outbound calls to public data APIs. Everything else ships inside one container image, which is what makes it portable to any cloud.

Data & persistence

SQLAlchemy ORM. The connection string is chosen at startup: **DATABASE_URL** (Postgres) → **TEROXX_DB_PATH** → local SQLite → /tmp fallback.

SIX TABLES

- **Client** — profile, domicile, currency, defaults, overrides
- **ClientLot** — individual holdings: ticker, qty, entry price/date
- **AllocationSnapshot** — frozen allocations behind each proposal
- **AdvisorAction** — full audit trail: who did what, when
- **ApiToken** — hashed tokens for the v1 API
- **Scenario** — saved A/B allocation comparisons

CONFIRM THE BACKEND

The app reads **DATABASE_URL** at startup. If it points at a managed Postgres (likely set in the Render dashboard), data is durable. If it is unset, the app falls back to SQLite on Render's free tier, whose disk is **ephemeral**.

Action: confirm the current backend in the Render dashboard. Client data is mostly test/throwaway, so this is not blocking — a fresh database is fine.

For Teroxx: deploy on a managed Postgres (RDS) with **DATABASE_URL** set, and enable backups. The code already supports it — no changes needed (step 2 of the runbook).

Access & security model

WHO CAN LOG IN

- Session-based login, no external identity provider
- Four users today: Jannick, Leonardo, Aleksander, Christopher
- Passwords stored salted + SHA-256 hashed in auth.py
- Emil (IT) has no login yet — add on handover

HOW SESSIONS ARE SIGNED

- Signed cookies via SESSION_SECRET (7-day expiry)
- Default dev secret MUST be overridden in production
- Middleware guards all UI routes; /health & static are open
- Regenerate the secret on migration (logs everyone out once)

MACHINE ACCESS (v1 API)

- Bearer tokens for /api/v1/* integrations
- Tokens stored hashed; plaintext shown once at creation
- Scoped (read / write / admin); revocable
- Lost tokens can't be recovered — mint a new one

Note for IT: this is lightweight internal auth, adequate for a small trusted desk. If the app is ever exposed more widely, front it with SSO / an identity-aware proxy and move users out of source code.

The proposal engine (and its dependencies)

One design rule keeps the three outputs identical: **the Word document is the single source of truth.**



WHAT THE CONTAINER MUST PROVIDE (all baked into the Docker image already)

- | | | | |
|---|--|---|---|
| LibreOffice Writer
converts DOCX → PDF (the soffice binary) | Pango / Cairo / HarfBuzz
rasterise SVG charts and shape text | Brand fonts
Söhne + Sometimes Times installed system-wide | TEROXX_VECTOR_CHARTS=0
PNG charts on small instances to avoid OOM |
|---|--|---|---|

Environment-variable reference

Everything configurable is an environment variable. This is the full contract between the app and its host.

VARIABLE	PURPOSE	ON MIGRATION	REQ
DATABASE_URL	Postgres connection string	Set on migration	required
SESSION_SECRET	Signs login cookies	Regenerate — secret	required
CMC_API_KEY	CoinMarketCap prices	Has public default	default ok
COINGECKO_API_KEY	CoinGecko fallback data	Has demo default	default ok
GOOGLE_SERVICE_ACCOUNT_JSON	Google Doc export credentials	Feature-off if unset	optional
GOOGLE_DRIVE_FOLDER_ID	Target Drive/Shared-Drive folder	Optional	optional
GOOGLE_IMPERSONATE_SUBJECT	Domain-wide delegation user	Optional	optional
GOOGLE_DOCS_SHARE	Share mode: none / domain / anyone	Default none	optional
TEROXX_VECTOR_CHARTS	Vector(1)/PNG(0) charts	0 on small hosts	default ok
PORT	Port the server binds to	Host usually sets	default ok

● required for a healthy production deploy ● needed only for Google Doc export ○ safe default exists

4

FOR IT

Standing it up on your cloud

- The migration in one picture
- A step-by-step runbook
- Hosting options & credentials

The migration in one picture

The app is a **single Docker container** plus a **Postgres database** plus a handful of **environment variables**. Anything that can run a container can run this — no code changes required.

The container

Built from the repo's Dockerfile. Bundles the app, LibreOffice and brand fonts. Nothing else to install.

The database

A managed PostgreSQL. Point the app at it with DATABASE_URL. Durable, backed up, no ops.

The secrets

~5 environment variables (Section 3 table). Set once in the host's dashboard.

The domain

Optional: map a Teroxx URL and TLS certificate. Most hosts do this in a click.

Step-by-step deployment runbook

1 Get the code

Clone from Teroxx GitLab. IT should own the repo and CI going forward.

2 Provision Postgres

Create a managed PostgreSQL; copy its connection string.

3 Provision the host

Create a container service pointing at the repo's Dockerfile.

4 Set env vars

DATABASE_URL, a fresh SESSION_SECRET, API keys, TEROXX_VECTOR_CHARTS.

5 Deploy

Build the image and start it. Tables auto-create on first boot.

6 Verify health

Hit /health (200 OK), log in, generate a test PDF proposal.

7 Map the domain

Point a Teroxx subdomain at the service; enable TLS.

8 Cut over

Update bookmarks, add Emil's login, suspend the old Render app.

Because the whole runtime is in the image, steps 3–5 are the same on every provider — only the dashboard differs. A companion runbook with exact commands accompanies this deck.

Where to run it — a recommendation

RECOMMENDED — AWS, aligned with Teroxx IT

IT has indicated **AWS with the EKS container platform** as Teroxx's standard, plus a managed **RDS Postgres**. The app is a single small, stateless service, so on EKS it is just one Deployment + Service + Ingress. If lower operational overhead is preferred, the same image runs unchanged on **AWS App Runner or ECS Fargate**. Exact setup to be finalised with IT.

OPTIONS AT A GLANCE

AWS EKS (Teroxx container standard) One Deployment + Service + Ingress, with RDS Postgres alongside. IT's named platform. Target — align with IT	AWS App Runner / ECS Fargate Same image, AWS-native, far less to operate than a full cluster. Lower-ops AWS option	PaaS (Render / Railway / Fly.io) Same model as today; render.yaml already in the repo. A quick bridge. Fastest lift-and-shift	Other clouds (Cloud Run / Container Apps) The identical container runs anywhere — noted for completeness. Portable if ever needed
--	--	---	---

Credentials & data transfer checklist

No live secrets are printed in this deck. Aleksander will hand these over through a secure channel (password manager / vault), not email or chat.

SECRETS TO TRANSFER & ROTATE

- **SESSION_SECRET** — generate a brand-new random value on the new host
- **CMC_API_KEY** — register Teroxx's own CoinMarketCap account & key; stop using the shared default
- **COINGECKO_API_KEY** — register Teroxx's own CoinGecko key; stop using the personal one
- **Google service account** — re-issue under Teroxx Workspace; share the Drive folder
- **Repo access** — transfer GitLab ownership to IT; retire the personal GitHub mirror
- **Render account** — hand over or rebuild; then delete the personal deployment

MOVING EXISTING DATA

Client data is **mostly test / throwaway**, so migration can start with a fresh Teroxx database — clients are re-entered as needed.

If anything must be kept: confirm the current backend, then dump it (Postgres or SQLite) and restore into the new Teroxx Postgres before cut-over. The schema is identical — SQLAlchemy creates it automatically.

Going forward: managed Postgres + automated daily backups keeps client data durable and recoverable.

5

RESEARCH + IT

Ownership & handover

- Who owns what
- First-week checklist
- Contacts & references

Who owns what, going forward

A clean split: Research owns the content and daily use; IT owns the platform it runs on.

RESEARCH — Jannick & Leonardo

- Day-to-day use: allocations, proposals, client reviews
- The model: universe, factor weights, tier rules, tilts
- Proposal content: narrative templates, EN/DE wording
- Deciding what changes are needed and prioritising them
- Adding / offboarding client records and advisor users

IT — Emil

- The cloud host, the database and its backups
- Secrets & environment variables (rotation, storage)
- The repo, CI/CD and deployments on push
- Domain, TLS, uptime and monitoring /health
- Applying code changes Research requests (or a dev)

First-week checklist & key references

FIRST WEEK — IN ORDER

- ☐ IT takes ownership of the GitLab repo
- ☐ Provision Postgres + container host (Section 4)
- ☐ Securely receive & set the secrets; rotate keys
- ☐ Deploy; verify /health, login, a test PDF proposal
- ☐ Map a Teroxx domain + TLS
- ☐ Add Emil's login; confirm Research can log in
- ☐ Cut over, then suspend the personal Render app
- ☐ Turn on database backups

KEY FILES IN THE REPO

Dockerfile the entire runtime recipe

render.yaml current deploy config & env keys

requirements.txt Python dependencies

app/db.py database models & connection logic

app/auth.py users & session secret

app/main.py all routes / the API surface

app/pdf/ the proposal (DOCX/PDF/GDoc) engine

CLAUDE.md in-repo notes & design decisions

handover/ this deck + companion runbook



Thank you — over to you

The app is small, self-contained and fully documented. Migrating it is a provision-and-deploy exercise, not a rewrite.

QUESTIONS DURING TRANSITION

Handover from

Aleksander Biesaga

aleksander.biesaga@teroxx.com

Research owners

**Jannick Broering · Leonardo
Lariera**

product & daily use

IT owner

Emil Jorgensen

platform & deployment